WNDRVR
SUPPORT NETWORK

VXWORKS 7 REAL-TIME PROCESS APPLICATION TUTORIAL - COMMAND LINE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Real-Time Process Application Tutorial - Command Line, SR0650

10 June 2020

1. REAL-TIME PROCESS APPLICATION TUTORIAL FROM THE COMMAND LINE

This VxWorks® 7 Real-Time Process Application Tutorial - Command Line, provides instructions on how to create, build, and run a Hello World real-time process (RTP) application from the command line.

RTP applications execute as real-time processes in user mode, each in their own protected memory space, which is separate from that of other RTP applications and from the kernel.

From the command line, you must base RTP projects on a VxWorks source build (VSB) project. The VSB produces binary kernel libraries an RTP project can use.

For this tutorial, the workflow is as follows:

1. Create and build a VSB.
2. Create a VIP based on the VSB and build the VIP.
3. Create an RTP application project based on the VSB.
4. Add code to the RTP project and build the project to obtain an RTP application.
5. Connect to the VxWorks simulator and boot VxWorks.
6. Load and run the RTP application.
7. Observe the results.

For more information, see Environment Variables for the VxWorks Development Tools.

2. CONFIGURING AND BUILDING VXWORKS SOURCE FROM THE COMMAND LINE

Before beginning any VxWorks development, generate the VxWorks kernel libraries by creating, configuring, and building a VxWorks source build (VSB) project.

About This Task

For this tutorial, create a VSB based on a board support package (BSP) for the VxWorks simulator.

Procedure

1. Start the wrtool utility.
   1. Open a command window and access the *installDir* **/workbench-4** directory.

      To view the startup command options for the wrtool utility, type **wrtool /?**.

      2. Type wrtool at the prompt to invoke the wrtool utility.

      The active workspace is set by the **WIND_WRTOOL_WORKSPACE** environment variable.

```
C:/WindRiver/workbench-4> wrtool
Active workspace is 'C:\WindRiver\workbench-4\workspace'.
WrTool, version 4.12.0
Copyright (c) 2013 - 2016 Wind River Systems, Inc.
workbench-4>
```

      You can also specify the active workspace with the -data command option.

```
C:/WindRiver/workbench-4> wrtool -data C:\WindRiver\workbench-4\workspace
Active workspace is 'C:\WindRiver\workbench-4\workspace'.
WrTool, version 4.12.0
Copyright (c) 2013 - 2016 Wind River Systems, Inc.
workbench-4>
```

      To view the list of command groups for the wrtool utility, type help.

      To view the list of commands for the wrtool session, type help builtin.

2. Access the active workspace directory.
3. Create a VSB project for the VxWorks simulator using the appropriate BSP for your host type.

Host OS	BSP
Windows	**vxsim_windows**_*x_x_x_x*
Linux	**vxsim_linux**_*x_x_x_x*

For this example, use the following syntax:

```
prj vsb create -bsp bspName -S name
```

To get help for the vsb create command option, type help prj vsb create.

To view the list of available BSPs, type prj vsb listBsps.

The following command creates a VSB for the Windows VxWorks simulator BSP version 1.0.0.0 and uses the -S option to accept all the default settings and not proceed with the interactive configuration process:

```
workspace> prj vsb create -bsp vxsim_windows_1_0_2_11 -S VxSimVSBwrtool2

Defaulting VSB to ILP32 to match BSP selected
Creating VSB Project ...

starting VSB configuration...

#
# configuration written to vsb.config
#

Checking layers required by BSP vxsim_windows_1_0_2_11.
Nothing to add all requested layers enabled

done

VSB Project created in : C:/WindRiver/workbench-4/workspace/VxSimVSBwrtool2

This VSB supports the following BSPs: vxsim_windows_1_0_2_11


workspace>
```

For this tutorial, accept the default configuration options and proceed directly to building the project.

Note: To view and configure the project, access the VSB project directory and use the following commands:

- To view the enabled project options, type prj vsb list.
- To view the available project options, type prj vsb listAll.
- To get help on configuring the project options, type help prj vsb config.

4. Build the VSB project using the prj build command and observe the build results.

```
workspace> prj build VxSimVSBwrtool
```

To get help for the build command option, type help prj build.

The project resides in the workspace folder. For example:

installDir **/workbench-4/workspace/VxSimVSBwrtool**

3. CONFIGURING AND BUILDING A VXWORKS KERNEL IMAGE FROM THE COMMAND LINE

To create a VxWorks kernel image, build VxWorks from binary components by creating a VxWorks image project (VIP) based on a VxWorks source build (VSB) project.

About This Task

For this tutorial, base the project on the **VxSimVSBwrtool** project.

Procedure

1. In the workspace directory, create a VIP project using the prj vip create command.

   For this example, use the following syntax:

   ```
   prj vip create -vsb vsbDir bspName toolName projDir
   ```

   Where *bspName* is the BSP for the VSB project and *toolName* is the toolchain.

   To get help for the vip create command option, type help prj vip create.

   The following command creates a VIP for the Windows VxWorks simulator named **VxSimVIPwrtool** that is based on the **VxSimVSBwrtool** project and uses the **gnu** tool chain:

   ```
   workspace> prj vip create -vsb VxSimVSBwrtool vxsim_windows_1_0_0_0 gnu VxSimVIPwrtool
   ...
   Project created in : C:/WindRiver/workspace/VxSimVIPwrtool/VxSimVIPwrtool.wpj
   ```

   For this tutorial, accept the default component and parameter configuration, and build the project.

   Note: To view and configure the project components, access the VIP project directory and use the following commands:

   - To view the included components, type prj vip component list.
   - To view the available components, type prj vip component list all.
   - To change the component configuration, use the prj vip component add and prj vip component remove commands.
   - To get help on configuring the VIP component parameter values, use the help prj vip parameter command.

2. To build the project, access the VIP directory and use the prj build command. For example:

   ```
   workspace> cd VxSimVIPwrtool
   VxSimVIPwrtool> prj build
   ```

   After the build completes, the image resides in the workspace folder. For example:

   installDir **/workbench-4/workspace/VxSimVIPwrtool/default/vxWorks**

WNDRVR

4. CREATING A REAL-TIME PROCESS PROJECT FROM THE COMMAND LINE

From the command line, a real-time process (RTP) application must be based on a VxWorks source build (VSB) project.

About This Task

For this tutorial, base the RTP project on the **VxSimVSBwrtool** project.

Procedure

1. Access the workspace directory and use the prj list command to list any existing VSB and VIP projects. For example:

```
myRTPwrtool >  cd ..
workspace> prj list
VxSimVIPwrtool (Wind River VxWorks 7 Image Project)
VxSimVSBwrtool (Wind River VxWorks 7 Source Build Project)
workspace>
```

2. Create an RTP project based on a VSB with the following syntax:

```
prj rtp create -vsb <vsbDir> <rtpProj>
```

   To get help for the rtp create command option, type help prj rtp create.

   The following command creates a project named **myRTPwrtool** based on the VSB project **VxSimVSBwrtool**:

```
workspace> prj rtp create -vsb VxSimVSBwrtool myRTPwrtool
```

   The project is created in the workspace folder. For example:

   installDir **/workbench-4/workspace/myRTPwrtool**

   A stub C source file named **rtp.c** is automatically added to the RTP project directory.

5. ADDING SOURCE CODE TO THE REAL-TIME PROCESS PROJECT

For this tutorial, add source code to the stub C source file and build the project to create a real-time process application.

About This Task

Add code to the **rtp.c** file to create an RTP application that displays `Hello World! From your RTP application...` on the console.

Procedure

1. Access the RTP project directory.
2. Using a text editor, add a call to printf( ) to the **rtp.c** file.

   The source code should be as follows:

```
#include <stdio.h>

int main (
    int    argc,        /* number of arguments */
    char * argv[]       /* array of arguments */
    ) {
    printf("Hello World!\nFrom your RTP application...\n");
    return 0;
}
```

3. Save the file.
4. Build the project and observe the successful completion of the build.

```
myRTPwrtool> prj build
```

5. Exit the wrtool utility:

```
myRTPwrtool> exit
```

6. CONNECTING TO THE VXWORKS SIMULATOR FROM THE COMMAND LINE

Connect to the VxWorks simulator from the host system.

Procedure

1. From the VxWorks installation directory, open a VxWorks development shell with the wrenv command.

   Use the appropriate command for your host.

Host OS	Method to Start a VxWorks Development Shell
Windows	`wrenv -p vxworks-7`
Linux	`./wrenv.sh -p vxworks-7:`

   For example:

```
C:/WindRiver> wrenv -p vxworks-7
```

   To get help for the wrenv command, type wrenv -help.

2. Start the simulator.
   1. Access the VIP image directory. For example:

```
C:/WindRiver> cd workbench-4/workspace/VxSimVIPwrtool/default
```

   2. Start the simulator with the following command:

```
C:/WindRiver/workbench-4/workspace/VxSimVIPwrtool/default> vxsim
```

   To get help for the vxsim command, type vxsim -help.

   The default vxsim boot device is a pass-through device (passDev) that does not require a host connection driver or the VxWorks network daemon *vxsimnetd*. No network device is created and no initialization is required.

   For more information on the VxWorks simulator, see the VxWorks 7 VxSim User's Guide.

   The VxSim0 window appears and displays the memory configuration and the VxWorks kernel shell prompt. For example:

```
VxSim0
Checking License...OK
Virtual  Base Address: 0x10000000
Virtual  Top  Address: 0x48000000    Virtual  Size: 0x38000000 (896Mb)
Physical Base Address: 0x10000000
Physical Top  Address: 0x18000000    Physical Size: 0x08000000 (128Mb)
Loading...
0x3782b0 + 0x3bb30

 _________            _________
 \77777777\          /77777777/
  \77777777\        /77777777/
   \77777777\      /77777777/
    \77777777\    /77777777/
     \77777777\   \777777/
      \77777777\   \77777/              VxWorks 7
       \77777777\   \777/
        \77777777\   \7/     Core Kernel version: 1.0.6.0
         \77777777\   -      Build date: Dec 11 2014 16:41:37
          \77777777\
           \7777777/         Copyright Wind River Systems, Inc.
            \77777/   -                1984-2014
             \777/   /7\
              \7/   /777\
               -   -------

             Board:  SIMNT board
     OS Memory Size: 127MB
   ED&R Policy Mode: Deployed

Loading symbol table from host:C:/vx7_nov18/vxworks/workbench-
4/workspace/myVipProj/default/vxWorks.sym ...done

-> |
```

To view the available commands, type help at the kernel shell prompt.

To reboot the instance, type reboot at the kernel shell prompt.

To terminate the simulation, close the VxSim0 window.

7. LOADING AND RUNNING THE REAL-TIME PROCESS APPLICATION FROM THE COMMAND LINE

After starting the VxWorks simulator, you can run the real-time process (RTP) application.

Procedure

1. From the VxWorks simulator shell, access the directory that contains the VxWorks executable (*.vxe). Use the appropriate method for your host.

Host OS	Method to Access the VxWorks Executable
Windows	Prefix the path with **host:** and enclose the entire path in quotes. For example: `-> cd "host:/c/WindRiver/workbench-4/workspace/myRTPwrtool/VxSimVSBwrtool_SIMNTgnu_C++2003/myRTPwrtool/Debug"`
Linux	Prefix the path with the name of the host on which the simulator is running followed by a colon. You must also enclose the entire path in quotes. For example: `-> cd "mars:/usr/WindRiver/workbench-4/workspace/myRTPwrtool/VxSimVSBwrtool_SIMLINUXgnu_C++2003/myRTPwrtool/Debug"`

2. Run the application using the RTP spawn command rtpSp.

   You must enclose the name of your executable in quotes. For example:

   ```
   -> rtpSp "myRTPwrtool.vxe"
   ```

   To get help for the **rtpSp** command, type rtpHelp.

   A process is an instance of a program and must be spawned to launch an application. When the application exits, the process terminates.

   The rtpSp command is a short form of the underlying rtpSpawn( ) function for launching RTP applications from the shell.

   The process loads and executes the application.

   ```
   Hello World!
   From your RTP application...
   ```

   When the application exits, the process terminates.

WNDRVR

8. WHERE TO NEXT?

After creating and adding code to a VxWorks application using command line, based on a tutorial, see the following steps to connect to your target, load your DKM and call C functions in the kernel.

1. To connect to a remote file system see *VxWorks 7 File Systems and I/O Programmer's Guide: Accessing Remote File Systems from VxWorks*.
2. To load a DKM object see *VxWorks 7 Kernel Shell User's Guide:Loading C Modules into the Kernel*.
3. To call C functions see *VxWorks 7 Kernel Shell User's Guide: Calling C Functions From the C Interpreter*.

For more information on making a target connection and debugging with the WRDBG (Wind River DeBuG) shell see the second Debug Session Example in *WRDBG Debug Shell Reference: Getting Started*

For more information on VxWorks projects, see the *VxWorks 7 Configuration and Build Guide* and the *Kernel Application and Real-Time Process Application Projects Guide*.

For more information on getting started with VxWorks 7, see the getting started documentation from the Wind River Support Network at Getting Started.